DhronAI

TECHNICAL ROUND · AI / ML ENGINEER

HIRING ASSIGNMENT · CONFIDENTIAL
VERSION 1.0 · APRIL 2026
TIME LIMIT: 72 HOURS
SUBMIT: GITHUB + DEMO VIDEO

LLM Fine-Tuning & Inference CLI

Train a local LLM on your JSONL data — then chat with it from the terminal

OVERVIEW

DhronAI builds AI-powered SaaS products for Indian businesses. Your task is to build **dhron-finetune** — a polished terminal CLI with two commands. **train** accepts a JSONL file, lets the user pick a model based on their hardware, configures hyperparameters, and runs LoRA/QLoRA fine-tuning locally. **chat** loads the saved adapter and opens an interactive streaming chat session.

END-TO-END FLOW

01 JSONL input › 02 Validate & stats › 03 Pick model › 04 Set hyperparams › 05 Train & save › 06 Chat / infer

THE TWO COMMANDS

train `dhron-finetune train --data ./train.jsonl --output ./output/`

Fine-tune on a JSONL dataset. Validates the file and shows dataset stats → hardware-aware model picker → interactive hyperparameter menu with defaults → LoRA/QLoRA training with live progress (epoch, step, loss, LR, ETA, GPU mem) → saves LoRA adapter and training report on completion. Ctrl+C must save a checkpoint gracefully without corrupting the output directory.

chat `dhron-finetune chat --model ./output/my-model-20260401/`

Load a saved adapter and chat interactively. Merges the LoRA adapter with the base model → multi-turn terminal chat loop → streams tokens as they are generated → displays model name and adapter path in the chat header → maintains full conversation history across all turns. Exit with Ctrl+C.

FUNCTIONAL REQUIREMENTS

Data Validation

- Accept `.jsonl` with `instruction`, `output`, optional `input`
- Show stats: sample count, avg tokens, file size
- Reject with a clear actionable error if invalid
- Warn (don't block) if fewer than 100 samples

Model Selection

- Auto-detect GPU / Apple MPS / CPU and available VRAM
- Show table: params, training VRAM, quantization type
- Warn if selected model may exceed available memory
- Registry extensible by adding one row — not a rewrite

Training UI

- Interactive hyperparameter menu with sensible defaults
- Validate all input ranges before training

Chat / Inference

- Load saved LoRA adapter and merge with base model
- Multi-turn terminal chat with token

ASSIGNMENT DETAILS

Time limit	72 hours
Language	Python 3.10+
Cloud deps	None — local
GPU required	No (CPU OK)
Submit via	GitHub + email

SUGGESTED MODELS

All trainable in <2 GB VRAM with QLoRA + gradient checkpointing + batch size 1.

MODEL	PARAMS	TRAIN VRAM
pythia-70m	70M	~0.5 GB
pythia-160m	160M	~0.8 GB
gpt2	124M	~1.0 GB
opt-125m	125M	~1.0 GB
bloom-560m	560M	~1.5 GB

HuggingFace IDs: `EleutherAI/pythia-70m`, `gpt2`, `facebook/opt-125m`, `bigscience/bloom-560m`

REQUIRED STACK

- transformers + PEFT
- bitsandbytes (QLoRA)
- trl (SFTTrainer)
- Rich + Typer
- torch (cuda / mps)
- pytest + pytest-cov

HYPERPARAMETER CONFIGURATION

PARAMETER	DEFAULT	VALID RANGE / OPTIONS	DESCRIPTION
TRAINING PARAMETERS			
num_epochs	3	1 – 20	Number of full passes over the training dataset
learning_rate	2e-4	1e-6 – 1e-3	Peak learning rate for the optimizer
batch_size	1	1 – 8	Per-device training batch size (keep low for <2GB)
gradient_accumulation	8	1 – 64	Steps before weight update (effective batch = batch × accum)
max_seq_length	256	64 – 1024	Maximum tokenized sequence length
warmup_ratio	0.03	0.0 – 0.3	Fraction of total steps used for LR warmup
weight_decay	0.01	0.0 – 0.1	L2 regularization strength
LORA PARAMETERS			
lora_r	8	4 – 64	LoRA rank — lower is faster, higher adds more capacity
lora_alpha	16	8 – 128	LoRA scaling factor (typically 2× rank)
lora_dropout	0.05	0.0 – 0.5	Dropout probability on LoRA adapter layers
STRATEGY			
training_type	qlora	lora, qlora	LoRA = full precision; QLoRA = 4-bit quantized base model
optimizer	adamw_torch	adamw_torch, adamw_8bit, sgd	Optimizer algorithm
lr_scheduler	cosine	cosine, linear, constant	Learning rate decay schedule shape

OUTPUT DIRECTORY STRUCTURE

```
output/<model>-<timestamp>/
  ├── adapter_model.safetensors  # LoRA weights
  ├── adapter_config.json        # LoRA config
  ├── tokenizer_config.json
  ├── training_log.jsonl         # per-step
metrics
  └── training_report.json       # final summary
```

JSONL DATASET FORMAT

```
// One JSON object per line
// instruction + output required, input optional

{
  "instruction": "Summarize the key points",
  "input": "Long article text here...",
  "output": "Key points: 1) ..."
}
```

DELIVERABLES

1. **GitHub Repository** — Clean modular structure with meaningful commits. No single-dump history. Include `sample_data/train.jsonl` (50–100 rows) so we can test immediately without preparing data.
2. **Working CLI** — Installs via `pip install -e .` . Both `train` and `chat` commands must run end-to-end. Use Rich + Typer or InquirerPy for all interactive parts — no bare argparse.
3. **README.md** — Setup instructions, usage examples for both commands, architecture decisions, and any known limitations or trade-offs made during the build.
4. **Unit Tests** — At least 60% coverage on non-ML code: data validation, config parsing, hyperparameter range checks, and model registry logic.
5. **Demo Recording** — 3–5 min screen recording (asciinema preferred, Loom acceptable): run `train` for 1 epoch → immediately run `chat` with the saved model → send a few messages and show the streaming output.

BONUS FEATURES (OPTIONAL — QUALITY OVER QUANTITY)

- Resume from checkpoint
- Train / val split + per-epoch val loss
- YAML config file support
- Chat format (messages[])
- System prompt in chat mode
- Export adapter to GGUF / Ollama

EVALUATION CRITERIA

CLI UX & Polish — 25%

Branded welcome screen, intuitive prompts, live progress display, clear validation errors, and consistent formatting throughout both commands.

train Correctness — 25%

Real LoRA/QLoRA training executes, adapter saves correctly, all hyperparameters are honoured, and Ctrl+C checkpoint works without corruption.

chat + Streaming — 20%

Adapter loads and merges correctly, tokens stream visibly to terminal, conversation history is maintained across all turns in the session.

Code Quality — 15%

Clean module separation, type hints on all public functions, no hardcoded paths or secrets, and a clean git commit history.

Tests & Docs — 10%

At least 60% test coverage on non-ML code, README is clear and complete, inline comments where logic is non-obvious.

Bonus Features — 5%

One well-implemented bonus scores higher than three half-finished ones. Implementation quality matters more than feature count.

AUTOMATIC DISQUALIFICATION

ANY OF THE FOLLOWING WILL IMMEDIATELY DISQUALIFY A SUBMISSION

- ✕ CLI fails to install or run on a clean environment
- ✕ Single-commit git history (no commit trail)
- ✕ Hardcoded API keys or secrets in the repository
- ✕ Fake or mocked training loop — no real model training
- ✕ chat command absent or non-functional
- ✕ Plagiarised code submitted without attribution